



UNIVERZITET U NOVOM SADU
PRIRODNO-MATEMATIČKI
FAKULTET
DEPARTMAN ZA MATEMATIKU
I INFORMATIKU



Web aplikacija za online prodaju društvenih igara razvijena u ASP.NET Core MVC tehnologiji

Praktični projekat iz predmeta Seminarski rad C

Nikolina Puač 267/21

Bogdan Kondić 259/21

Novi Sad, 2026

Sadržaj

1. Zadatak rada.....	2
2. Koncept rešenja.....	3
3. Odabrane tehnologije.....	4
ASP.NET Core MVC (.NET 10).....	4
Entity Framework Core i Microsoft SQL Server.....	4
ASP.NET Core Identity.....	4
Sesija (ASP.NET Core Session / Distributed Memory Cache).....	4
Tailwind CSS v4 (preko @tailwindcss/cli).....	4
Tailwind Plus Elements.....	4
4. Detalji implementacije za ključne komponente sistema.....	5
4.1 Autentifikacija, autorizacija i uloge korisnika.....	5
4.2 Modeli podataka i EF Core.....	6
4.3 Korpa za kupovinu i rad sa sesijom (Session).....	7
Podešavanje sesije.....	7
Serijalizacija korpe u sesiji.....	8
CartController - čuvanje i čitanje korpe.....	8
Prepoznavanje sesija korisnika (Kako se održava nezavisnost korpi).....	8
Ažuriranje i praćenje zaliha (Stock Quantity).....	9
Prelazak iz sesije u bazu: Checkout i PlaceOrder.....	9
4.4 Integracija Tailwind CSS-a u build proces.....	10
Setup NPM okruženja.....	10
Ulazni fajl i Tailwind v4 konfiguracija.....	10
MSBuild target - automatsko generisanje CSS-a.....	11
Povezivanje sa Layout-om i korišćenje utility klasa.....	11
Tailwind Plus Elements - interaktivnost bez sopstvenog JavaScript-a.....	11
Responzivnost menija.....	12
4.5 Administratorski panel.....	13
5. Primer funkcionisanja rešenja.....	14
5.1 Gost pregleda katalog i puni korpu.....	14
5.2 Potvrda porudžbine (checkout).....	14
5.3 Istorija porudžbina.....	14
5.4 Administracija.....	14
6. Zaključak.....	15
7. Literatura.....	16

1. Zadatak rada

Zadatak seminarskog rada je bio da se osmisli i realizuje desktop, mobilna ili web aplikacija. Mi smo izabrali web aplikaciju tipa e-commerce (online prodavnica) realizovana korišćenjem ASP.NET Core MVC tehnologije, sa bazom podataka, sistemom korisničkih naloga i administratorskim delom sistema. Za konkretan proizvod odabrana je prodavnica društvenih igara, jer domen ima prirodnu podelu na kategorije, proizvode i porudžbine, što dobro ilustruje sve tražene funkcionalnosti.

Konkretno, aplikacija omogućava sledeće funkcionalnosti:

1. Pregled svih dostupnih proizvoda, sa mogućnošću filtriranja po kategoriji i pretrage po nazivu/opisu.
2. Pregled detalja pojedinačnog proizvoda (cena, opis, stanje na zalihama, kategorija).
3. Korpu za kupovinu (shopping cart) koja radi i za neprijavljene (goste) korisnike.
4. Registraciju i prijavu korisnika (autentifikaciju), uz mogućnost razlikovanja običnog korisnika od administratora (autorizacija po ulogama).
5. Naručivanje (checkout) koje je dozvoljeno samo prijavljenim korisnicima, sa unosom adrese za dostavu.
6. Pregled istorije sopstvenih porudžbina za prijavljenog korisnika.
7. Administratorski panel za: dodavanje novih proizvoda, pregled i promenu statusa svih porudžbina u sistemu, pregled/banovanje korisnika i pregled statistike prodaje po proizvodima.
8. Konzistentan i moderan korisnički interfejs, uz korišćenje savremenog CSS pristupa (Tailwind CSS) umesto ručno pisanog CSS-a.

2. Koncept rešenja

Rešenje je organizovano po standardnom MVC (Model-View-Controller) obrascu koji ASP.NET Core podržava. Aplikacija je podeljena na nekoliko logičkih celina:

1. **Modeli (Models)** - klase koje opisuju podatke (Product, Category, Order, OrderItem, ApplicationUser)
2. **ViewModel** - klase namenjene isključivo prikazu (CartItemViewModel, AdminDashboardViewModel, ProductStatViewModel).
3. **Kontroleri (Controllers)** - ProductController, CartController, OrderController i AdminController, koji obrađuju HTTP zahteve, komuniciraju sa bazom preko EF Core i biraju odgovarajući View.
4. **Views (Razor)** - .cshtml fajlovi koji prikazuju podatke korisniku, stilizovani Tailwind CSS klasama.
5. **Area "Identity"** - Razor Pages koje dolaze iz ASP.NET Core Identity biblioteke (registracija, prijava, odjava), prilagođene izgledu ostatka sajta.
6. **Sloj za pristup podacima** - ApplicationDbContext (Entity Framework Core) koji se preko migracija sinhronizuje sa MS SQL Server bazom.
7. **Sesija (Session)** - poseban mehanizam koji se koristi isključivo za privremeno čuvanje korpe za kupovinu, van baze podataka.

Veze između komponenti prate klasičan MVC tok: korisnikov zahtev iz browsera stiže do odgovarajućeg kontrolera (rutiranje je podešeno tako da je podrazumevana početna stranica Product/Index). Kontroler po potrebi čita/upisuje podatke preko ApplicationDbContext-a, priprema model (ili ViewModel) i prosleđuje ga pripadajućem Razor View-u koji generiše HTML stilizovan Tailwind CSS klasama.

Poseban slučaj predstavlja korpa za kupovinu. Njeni podaci ne idu direktno u bazu već se, dok korisnik "kupuje", drže u ASP.NET Core sesiji, a tek prilikom potvrde porudžbine (checkout) trajno se upisuju u bazu kao Order i OrderItem zapisi. Ovaj deo koncepta detaljnije je opisan u poglavlju 4.3.

Autorizacija je rešena na nivou uloga: svaki korisnik pri registraciji dobija podrazumevanu ulogu, dok administratorski nalog (kreiran automatski pri prvom pokretanju aplikacije, u Program.cs) dobija ulogu "Admin". AdminController i njegove akcije su zaštićene atributom [Authorize(Roles = "Admin")], dok je pristup checkout-u i istoriji porudžbina ograničen na prijavljene korisnike atributom [Authorize].

3. Odabrane tehnologije

ASP.NET Core MVC (.NET 10)

Predstavlja glavni framework aplikacije. Odabran je zato što nudi kompletnu, integrisanu podršku za rutiranje, kontrolere, Razor View engine, dependency injection i konfiguraciju servisa (npr. session, autentifikacija) na jednom mestu, bez potrebe za dodatnim bibliotekama.

Entity Framework Core i Microsoft SQL Server

EF Core je korišćen kao ORM (Object-Relational Mapper) koji omogućava rad sa bazom kroz obične C# klase (Product, Order, Category...) umesto ručno pisanog SQL-a. Odabran je zbog Code-First pristupa i migracija (Migrations), koje omogućavaju da se šema baze verzioniše zajedno sa kodom. SQL Server je odabran jer se prirodno integriše sa EF Core paketom Microsoft.EntityFrameworkCore.SqlServer i Visual Studio okruženjem.

ASP.NET Core Identity

Korišćen je za kompletno upravljanje korisničkim nalogima, registraciju, prijavu, heširanje lozinki, zaključavanje (lockout) naloga i upravljanje ulogama (RoleManager, UserManager) potrebnim za razlikovanje običnog korisnika od administratora. Odabran je umesto ručne implementacije autentifikacije zato što je bezbednosno proveren, ugrađen u ASP.NET Core i lako proširiv (npr. ApplicationUser nasleđuje IdentityUser).

Sesija (ASP.NET Core Session / Distributed Memory Cache)

Ugrađeni mehanizam sesije korišćen je isključivo za čuvanje korpe za kupovinu gostujućih i prijavljenih korisnika pre nego što se porudžbina potvrdi. Odabran je zato što ne zahteva dodatnu tabelu u bazi za nešto što je suštinski privremeno stanje, a korisniku omogućava da puni korpu bez obaveze da se odmah registruje.

Tailwind CSS v4 (preko @tailwindcss/cli)

Tailwind CSS je odabran kao utility-first CSS framework, umesto pisanja sopstvenih CSS klasa i selektora, izgled elemenata se definiše direktno u HTML/Razor markupu kroz gotove, sitne utility klase (npr. flex, px-4, rounded-md, text-gray-300). Ovo je značajno ubrzalo izradu korisničkog interfejsa u odnosu na pisanje CSS-a od nule, i obezbedilo vizuelnu konzistentnost (iste boje, razmake i tipografiju) kroz ceo sajt bez posebnog dogovaranja unutar tima. Odabrana je verzija 4, koja uvodi CSS-first konfiguraciju i CLI alat koji generiše finalni CSS fajl u build koraku. Ovaj deo je detaljno opisan u poglavlju 4.4, jer je to bio jedan od fokusa rada.

Tailwind Plus Elements

Uz Tailwind CSS, korišćena je i biblioteka Tailwind Plus Elements (učitana kao ES modul preko CDN-a u _Layout.cshtml) koja dodaje gotove, pristupačne (accessible) HTML elemente. Njima

je realizovan padajući meni korisničkog profila i meni za mobilne uređaje bez pisanja sopstvenog JavaScript koda.

4. Detalji implementacije za ključne komponente sistema

U ovom poglavlju su opisane ključne komponente sistema. Poseban naglasak je stavljen na implementaciju korpe za kupovinu preko sesije (poglavljje 4.3) i na integraciju Tailwind CSS-a u build proces aplikacije (poglavljje 4.4).

4.1 Autentifikacija, autorizacija i uloge korisnika

Korisnički nalozi se čuvaju pomoću klase ApplicationUser koja nasleđuje IdentityUser iz ASP.NET Core Identity biblioteke i dodaje kolekciju porudžbina korisnika (Orders):

```
public class ApplicationUser : IdentityUser
{
    0 references
    public List<Order> Orders { get; set; } = new();
}
```

Uloge se koriste da bi se razlikovao običan korisnik od administratora. Prilikom prvog pokretanja aplikacije (u Program.cs), automatski se kreira uloga "Admin" i jedan administratorski nalog ukoliko još ne postoje, time se izbegava ručno podešavanje administratora nad svakom novom instancom baze:

```
if (!await roleManager.RoleExistsAsync(adminRole))
{
    await roleManager.CreateAsync(new IdentityRole(adminRole));
}

var adminUser = await userManager.FindByEmailAsync(adminEmail);
if (adminUser == null)
{
    adminUser = new ApplicationUser
    {
        UserName = adminEmail,
        Email = adminEmail,
        EmailConfirmed = true
    };

    await userManager.CreateAsync(adminUser, adminPassword);
}
```

Pristup je zatim ograničen deklarativno, atributima na kontrolerima i akcijama: [Authorize] zahteva bilo kog prijavljenog korisnika (npr. za Checkout i istoriju porudžbina), dok [Authorize(Roles = "Admin")] na celom AdminController-u ograničava ceo administratorski deo sistema samo na naloge sa ulogom "Admin". Dodatno, u Program.cs je dodat middleware koji

odmah odjavljuje korisnika čiji je nalog banovan (LockoutEnd u budućnosti), umesto da se čeka isticanje postojeće sesije prijave.

```
app.Use(async (context, next) =>
{
    if (context.User.Identity?.IsAuthenticated == true)
    {
        var userManager = context.RequestServices.GetRequiredService<userManager<ApplicationUser>>();
        var user = await userManager.GetUserAsync(context.User);
        if (user != null && user.LockoutEnd.HasValue && user.LockoutEnd > DateTimeOffset.UtcNow)
        {
            var signInManager = context.RequestServices.GetRequiredService<SignInManager<ApplicationUser>>();
            await signInManager.SignOutAsync();
            context.Response.Redirect("/Identity/Account/Login");
            return;
        }
    }

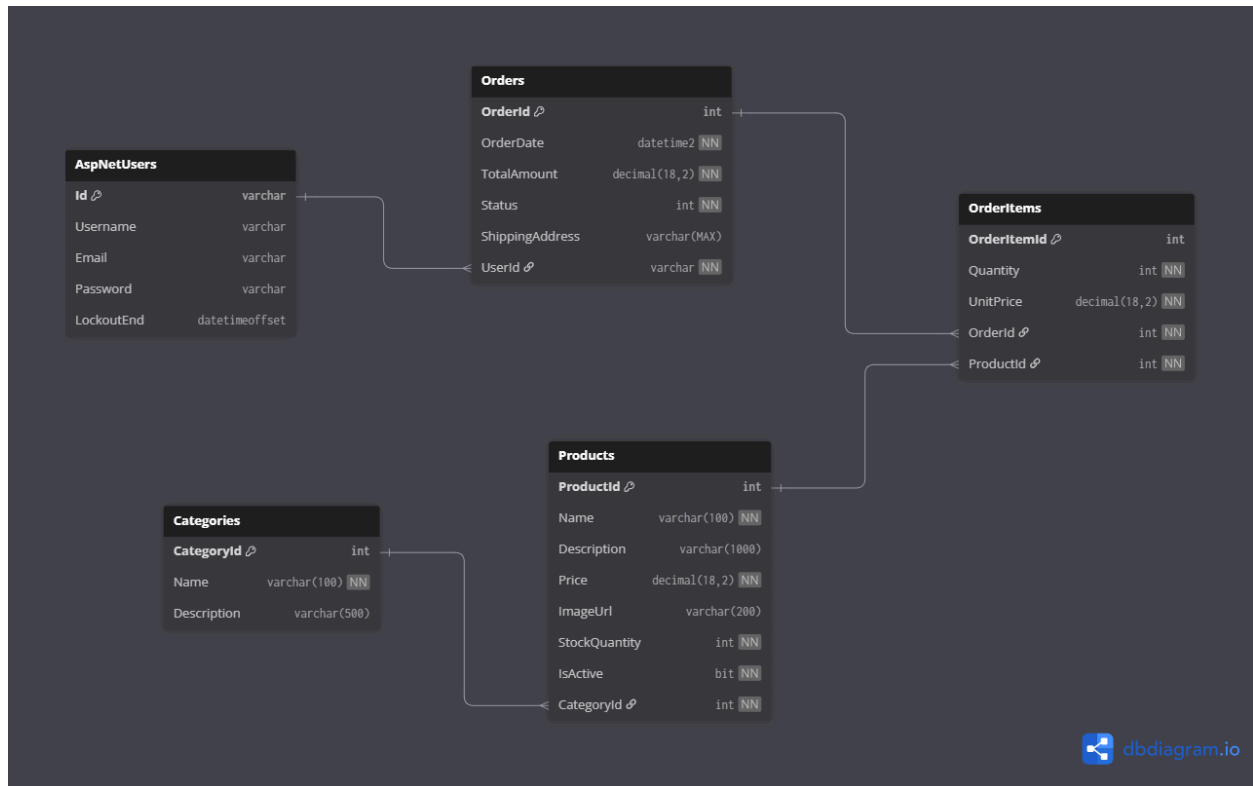
    await next();
});
```

4.2 Modeli podataka i EF Core

Domen aplikacije je modelovan korišćenjem Code-First pristupa kroz Entity Framework Core. To znači da je kompletna šema baze podataka generisana isključivo na osnovu C# klasa (modela) i EF Core migracija (folder Migrations), bez ručnog pisanja SQL skripti.

Sistem se oslanja na pet glavnih entiteta i njihove međusobne relacije:

1. **ApplicationUser:** Nasleđuje ugrađenu IdentityUser klasu i predstavlja registrovane kupce i administratore. Jedan korisnik može imati više porudžbina (relacija 1-N sa tabelom Order).
2. **Category:** Služi za klasifikaciju asortimana. Povezana je sa proizvodima relacijom 1-N. Jedna kategorija može sadržati više proizvoda.
3. **Product:** Centralni entitet aplikacije koji čuva naziv, cenu, sliku i stanje zaliha (StockQuantity). Svaki proizvod poseduje strani ključ ka svojoj kategoriji (CategoryId).
4. **Order:** Predstavlja kreiranu porudžbinu. Čuva datum, ukupnu cenu, adresu dostave i vezu ka korisniku koji je izvršio kupovinu. Status porudžbine je modelovan enumeracijom (Pending, Processing, Shipped, Delivered, Cancelled), što administratoru omogućava da menja tok porudžbine kroz Admin panel (poglavlje 4.5).
5. **OrderItem:** Predstavlja tabelu koja spaja porudžbine i proizvode. Umesto direktne N-M veze između tabela Order i Product, OrderItem razrađuje ovu vezu beležeći tačnu količinu i cenu u trenutku kupovine. Ovo garantuje da se istorija porudžbina neće pokvariti ukoliko se cena proizvoda u Product tabeli kasnije promeni.



Slika 1: Šema baze podataka

4.3 Korpa za kupovinu i rad sa sesijom (Session)

Ovo je jedna od centralnih funkcionalnosti sistema: korpa mora da radi i za korisnike koji nisu prijavljeni, a da se tek pri potvrđi porudžbine zahteva prijava. Zbog toga korpa nije modelovana kao tabela u bazi, već kao objekat koji živi u ASP.NET Core sesiji, vezanoj za sesijski kolačić (cookie) korisnikovog pregledača.

Podešavanje sesije

Sesija se aktivira u Program.cs dodavanjem distribuiranog memorijskog keša (koji čuva sadržaj sesije na serveru) i konfigurisanjem trajanja i kolačića same sesije:

```

builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});
  
```

Bitno je da se `app.UseSession()` pozove pre `app.UseAuthorization()`, i pre mapiranja ruta, kako bi sesija bila dostupna u svakom zahtevu.

Serijalizacija korpe u sesiji

Ugrađeni `ISession` interfejs ume da čuva samo stringove, brojeve i bajtove, ali ne i složene objekte poput liste stavki korpe. Zato je napravljeno par generičkih ekstenzionih metoda (`Extensions/SessionExtensions.cs`) koje ceo objekat serijalizuju u JSON pre upisa u sesiju, i deserijalizuju nazad pri čitanju:

```
public static class SessionExtensions
{
    1 reference
    public static void SetObject<T>(this ISession session, string key, T value)
    {
        var json = JsonSerializer.Serialize(value);
        session.SetString(key, json);
    }

    1 reference
    public static T? GetObject<T>(this ISession session, string key)
    {
        var json = session.GetString(key);
        if (string.IsNullOrEmpty(json))
        {
            return default;
        }
        return JsonSerializer.Deserialize<T>(json);
    }
}
```

Ovaj pristup je izabran jer je generičan (radi za bilo koji tip, ne samo za korpu) i jer izbegava ručno pisanje serijalizacije u svakom kontroleru posebno.

CartController - čuvanje i čitanje korpe

`CartController` koristi konstantu `CartSessionKey` ("ShoppingCart") kao ključ pod kojim se korpa čuva u sesiji, i dve privatne pomoćne metode koje enkapsuliraju pristup sesiji:

```
private const string CartSessionKey = "ShoppingCart";

3 references
private void SaveCart(CartViewModel cart)
{
    HttpContext.Session.SetObject(CartSessionKey, cart);
}

7 references
private CartViewModel GetCart()
{
    return HttpContext.Session.GetObject<CartViewModel>(CartSessionKey) ?? new CartViewModel();
}
```

Prepoznavanje sesija korisnika (Kako se održava nezavisnost korpi)

Praćenje stanja za aplikacije preko web-a je podrazumevano "stateless" (server ne pamti korisnika između dva HTTP zahteva). Oslanjanje na sesiju to rešava. Kada se pozovu `GetCart` ili `SaveCart` funkcije, ASP.NET Core u pozadini identifikuje korisnika kroz jedinstveni "Session ID" koji se prosleđuje upakovan u session cookie. Bez ikakvog dodatnog pisanja koda za tu namenu obezbeđena je potpuna nezavisnost, čak i ako više neprijavljenih korisnika istovremeno dodaje proizvode pod istim string ključem ("ShoppingCart"), oni ne gaze jedni drugima podatke, već svako pristupa strogo svom delu memorije.

Ažuriranje i praćenje zaliha (Stock Quantity)

Svaka akcija koja menja korpu (AddToCart, UpdateQuantity, RemoveFromCart) prati isti obrazac: učitava se trenutna korpa iz sesije, izmeni se lista stavki (CartItem), uporedo se ažurira StockQuantity odgovarajućeg proizvoda u bazi (da zalihe uvek odražavaju ono što je u korpama), i na kraju se izmenjena korpa ponovo upiše u sesiju:

```
var cart = GetCart();
var existingItem = cart.Items.FirstOrDefault(i => i.ProductId == productId);

if (existingItem != null)
{
    existingItem.Quantity += quantity;
}
else
{
    cart.Items.Add(new CartItem
    {
        ProductId = product.ProductId,
        ProductName = product.Name,
        Price = product.Price,
        Quantity = quantity,
        ImageUrl = product.ImageUrl
    });
}
product.StockQuantity -= quantity;
await _context.SaveChangesAsync();
SaveCart(cart);
return RedirectToAction("Index");
```

CartItem je označen atributom [NotMapped]. To je klasa koja postoji samo radi prikaza i čuvanja u sesiji, i nikada se direktno ne upisuje u bazu kao posebna tabela.

Prelazak iz sesije u bazu: Checkout i PlaceOrder

Akcija Checkout je označena atributom [Authorize], što znači da je neprijavljeni korisnik automatski preusmeren na stranicu za prijavu čim pokuša da potvrdi porudžbinu, do tog trenutka korpa je u potpunosti postojala samo u sesiji. Kada prijavljeni korisnik potvrdi porudžbinu (PlaceOrder), sadržaj korpe iz sesije se pretvara u trajne Order i OrderItem zapise u bazi, a sesija se zatim briše:

```
var order = new Order
{
    UserId = userId,
    TotalAmount = cart.TotalPrice,
    ShippingAddress = shippingAddress,
    Status = Order.OrderStatus.Pending,
    OrderDate = DateTime.Now
};

foreach(var item in cart.Items) {...}

_context.Orders.Add(order);
await _context.SaveChangesAsync();
HttpContext.Session.Remove(CartSessionKey);
return RedirectToAction("OrderConfirmation", new { orderId = order.OrderId });
```

Ovakav dizajn ima dve praktične prednosti: korisnik može slobodno da razgleda katalog i puni korpu bez ikakve obaveze registracije, a baza podataka ne sadrži "napuštene" ili nepotvrđene porudžbine, jer se zapis u tabeli Orders kreira tek kada je porudžbina zaista potvrđena.

4.4 Integracija Tailwind CSS-a u build proces

Za razliku od pristupa gde bi se Tailwind CSS kompajlirao i pokretao ručno sa strane tokom razvoja, u ovom projektu je on u potpunosti integrisan u standardni .NET build. To znači da se finalni stilovi osvežavaju automatski pri svakom pokretanju projekta iz Visual Studio-a.

Setup NPM okruženja

ASP.NET Core projekti po podrazumevanom ponašanju ne uključuju JavaScript/NPM alate za build procese. Da bi se omogućila upotreba Tailwind v4 CLI alata, projekat je prvo morao biti inicijalizovan kao Node.js repozitorijum. To je postignuto pokretanjem komande za definisanje zavisnosti u root folderu u terminalu:

```
npm -init y
```

Nakon toga instalirani su osnovni alati potrebni za Tailwind:

```
npm install -D tailwindcss @tailwindcss/cli
```

Ovo je rezultiralo generisanjem package.json i package-lock.json fajlova koji beleže da projekat sada zvanično zavisi od dotičnih alata:

```
"dependencies": {  
  "@tailwindcss/cli": "^4.3.0",  
  "tailwindcss": "^4.3.0"  
}
```

Ulazni fajl i Tailwind v4 konfiguracija

Projekat koristi Tailwind CSS u verziji 4, koja napušta poseban tailwind.config.js fajl iz verzije 3 u korist tzv. CSS-first konfiguracije. Zbog toga je ceo "ulazni" stylesheet (wwwroot/css/site.css) sveden na samo jednu liniju:

```
@import "tailwindcss";
```

Ova linija uvozi kompletan Tailwind CSS okvir (bazne stilove, komponente i sve utility klase), a alat zatim sam analizira sve .cshtml fajlove u projektu i u izlazni fajl ubacuje samo one utility klase koje su zaista upotrebljene u markupu (npr. flex, rounded-md, bg-gray-800), čime se finalni CSS drži što je moguće manjim.

MSBuild target - automatsko generisanje CSS-a

Ključni deo integracije nalazi se u Project.csproj, gde je definisan poseban MSBuild target "Tailwind" koji se izvršava pre svakog build-a i pre pripreme za publish (BeforeTargets="Build;PrepareForPublish"). On prvo instalira npm pakete, a zatim pokreće Tailwind CLI koji iz ulaznog fajla generiše minifikovani izlazni CSS:

```
<Target Name="Tailwind" BeforeTargets="Build;PrepareForPublish">
  <Exec Command="npm install" />
  <Exec Command="npx @tailwindcss/cli -i ./wwwroot/css/site.css -o ./wwwroot/css/output.css --minify" />
  <ItemGroup>
    <Content Include="wwwroot/css/output.css" CopyToOutputDirectory="Always" />
  </ItemGroup>
</Target>
```

Povezivanje sa Layout-om i korišćenje utility klasa

Generisani output.css fajl se učitava u zajedničkom _Layout.cshtml-u, uz asp-append-version koji dodaje hash verzije fajla u URL, tako da pregledač uvek učitava najnoviju verziju CSS-a nakon izmena:

```
<link rel="stylesheet" href="~/css/output.css" asp-append-version="true"/>
```

Utility klase se zatim koriste direktno u markupu, bez posebnih CSS fajlova po komponenti. Na primer, navigaciona traka kombinuje klase za raspored (flex, items-center, justify-between), boje (bg-gray-800, text-gray-300) i interaktivna stanja (hover:bg-white/5, hover:text-white):

```
<nav class="bg-gray-800">
  <div class="mx-auto max-w-7xl px-4 sm:px-6 lg:px-8">
    <div class="flex h-16 items-center justify-between">
      <div class="flex items-center">
        <div class="hidden md:block">
          <div class="ml-10 flex items-baseline space-x-4">
            <a asp-controller="Product" asp-action="Index">
```

Prefiksi poput sm: i lg: obezbeđuju responzivnost (npr. hidden md:block sakriva element na malim ekranima, a prikazuje ga od srednje veličine ekrana naviše), tako da isti markup radi i na mobilnim i na desktop prikazima, bez dodatnih media query pravila.

Tailwind Plus Elements - interaktivnost bez sopstvenog JavaScript-a

Pored čistog stilizovanja, u _Layout.cshtml je uključena i biblioteka Tailwind Plus Elements (učitana kao ES modul preko CDN-a), koja dodaje gotove HTML elemente sa ugrađenim ponašanjem.

```
<script src="https://cdn.jsdelivr.net/npm/@tailwindplus/elements@1" type="module"></script>
```

Ovim se izbegava pisanje ručnog JavaScript koda za otvaranje/zatvaranje menija. Ponašanje dolazi iz biblioteke, dok Tailwind CSS klase kontrolišu isključivo izgled (pozicioniranje, animacije prelaza poput transition i data-closed:opacity-0, senke, zaobljene ivice).

Responzivnost menija

Kako bi se osiguralo dobro iskustvo na svim uređajima, glavna navigacija koristi pristup baziran na ugrađenim funkcionalnostima Tailwind CSS. Aplikacija detektuje širinu ekrana klijenta i automatski prikazuje ili skriva određene elemente, što kreira dve različite opcije navigacije:

1. **Desktop opcija:** Na standardnim monitorima i većim tabletima svi linkovi ka stranicama i ikonice korisničkog profila i korpe su vidljivi i horizontalno poređani na vrhu ekrana. To se postiže Tailwind klasom `hidden md:block`, koja sistemu govori: "Sakrij ovaj deo koda (hidden) odmah u startu, ali ako je širina ekrana veća od 'medium' uređaja (md), onda ga prikaži (block)".

```
<div class="hidden md:block">
  <div class="ml-10 flex items-baseline space-x-4">
    <a asp-controller="Product" asp-action="Index" class="rounded-md px-3 py-2">
    <a asp-controller="Order" asp-action="Index" class="rounded-md px-3 py-2">
    @if (User.IsInRole("Admin"))
    {
      <a asp-controller="Admin" asp-action="Index" class="rounded-md px-3 py-2">
    }
  </div>
</div>
```

2. **Mobilna opcija:** Na mobilnim telefonima nema dovoljno prostora za horizontalni ispis svih linkova. Zato se gornji linkovi skrivaju automatski, a umesto njih se na ekranu pojavljuje tkz. Mobile menu button. Za ovo dugme je korišćen Tailwind Plus Elements sa specijalnim atributima `command` i `commandfor="mobile-menu"`. Kada korisnik pritisne to dugme, okida se HTML ponašanje koje otvara (otkriva) element sa ID-jem `mobile-menu` (koji je sakriven preko `<el-disclosure id="mobile-menu" hidden>`). Zatim se linkovi ka proizvodima i profilu otvaraju vertikalno naniže.

```
<div class="--mr-2 flex md:hidden">
  <!-- Mobile menu button -->
  <button type="button" command="--toggle" commandfor="mobile-menu" class="relative inline-flex items-center justify-center rounded-md p-2 text-gray-400 hover:bg-white/5">
    <span class="absolute -inset-0.5"></span>
    <span class="sr-only">Open main menu</span>
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5" data-slot="icon" aria-hidden="true" class="size-6 in-aria-expanded:hidden">
      <path d="M3.75 6.75h16.5M3.75 12h16.5M3.75 17.25h16.5" stroke-linecap="round" stroke-linejoin="round" />
    </svg>
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5" data-slot="icon" aria-hidden="true" class="size-6 not-in-aria-expanded:hidden">
      <path d="M6 18 18 6M6 6 12 12" stroke-linecap="round" stroke-linejoin="round" />
    </svg>
  </button>
</div>
```

4.5 Administratorski panel

Ceo AdminController je zaštićen atributom [Authorize(Roles = "Admin")], pa je nedostupan običnim korisnicima. Nudi četiri glavne celine:

1. **Dashboard (Index)** - agregatni pregled stanja sistema (broj korisnika, broj banovanih korisnika, ukupan i pending/processing broj porudžbina, ukupan prihod), izračunat direktno preko EF Core LINQ upita (CountAsync, SumAsync).
2. **Dodavanje proizvoda (CreateProduct)** - forma koja prima ili spoljašnji URL slike ili fajl koji se čuva u wwwroot/images/products uz generisanje jedinstvenog imena fajla (Guid).
3. **Upravljanje porudžbinama (Orders / UpdateOrderStatus)** - pregled svih porudžbina u sistemu (ne samo trenutnog korisnika) i promena statusa porudžbine (Pending -> Processing -> Shipped -> Delivered, ili Cancelled).
4. **Upravljanje korisnicima (Users / ToggleBan)** - banovanje/odbanovanje korisničkog naloga korišćenjem ugrađenog Identity lockout mehanizma (SetLockoutEndDateAsync), uz eksplicitnu proveru da administrator ne može da banuje samog sebe niti drugog administratora.
5. **Statistika prodaje (Stats)** - grupisanje stavki porudžbina po proizvodu (GroupBy) radi prikaza ukupno prodane količine, ostvarenog prihoda i broja porudžbina po svakom proizvodu.

Banovanje se ne implementira kao posebno polje u bazi, već ponovnom upotrebom postojećeg Identity mehanizma za zaključavanje naloga (LockoutEnd postavljen u daleku budućnost predstavlja trajni ban), čime se izbegava dodatna migracija i duplirana logika.

5. Primer funkcionisanja rešenja

U nastavku je opisan tipičan scenario korišćenja aplikacije, kroz koji se ilustruju ključne funkcionalnosti opisane u prethodnim poglavljima.

5.1 *Gost pregleda katalog i puni korpu*

Korisnik koji nije prijavljen otvara aplikaciju i automatski dolazi na Product/Index (podrazumevana ruta). Tu može da filtrira proizvode po kategoriji (Strategy Games, Family Games, Card Games) preko parametra categoryId, ili da pretraži proizvode po nazivu/opisu preko parametra searchString. Oba filtera se kombinuju u istom upitu u ProductController.Index.

Klikom na proizvod, korisnik vidi Details stranicu sa opisom, cenom i stanjem zaliha. Dodavanjem proizvoda u korpu (dugme "Add to cart") korpa se, kao što je opisano u 4.3, čuva u sesiji. Broj stavki u korpi je odmah vidljiv, bez ikakve potrebe za registracijom.

5.2 *Potvrda porudžbine (checkout)*

Kada gost pokuša da otvori Cart/Checkout, [Authorize] atribut ga automatski preusmerava na stranicu za prijavu (Areas/Identity/Pages/Account/Login). Nakon prijave (ili registracije, ako korisnik nema nalog), korisnik se vraća na Checkout stranicu, gde vidi pregled svoje korpe i unosi adresu za dostavu. Potvrdom porudžbine (PlaceOrder), sistem iz sesijske korpe kreira Order i pripadajuće OrderItem zapise u bazi, korpa se briše iz sesije, a korisnik se preusmerava na OrderConfirmation stranicu sa detaljima upravo kreirane porudžbine.

5.3 *Istorija porudžbina*

Prijavljeni korisnik može u svakom trenutku da ode na Order/Index i vidi spisak isključivo svojih porudžbina (filtriranih po UserId iz trenutne autentifikacije), kao i detalje svake porudžbine (Order/Details) sa stavkama i cenama.

5.4 *Administracija*

Administrator se prijavljuje istim login formularom, ali mu se, zahvaljujući proveru User.IsInRole("Admin") u _Layout.cshtml, u navigaciji dodatno prikazuje link ka Admin panelu. Tu može da: doda novi proizvod, pregleda dashboard sa ključnim pokazateljima (broj korisnika, porudžbina, prihod), promeni status bilo koje porudžbine u sistemu, banuje korisnika i pregleda statistiku najprodavanijih proizvoda.

6. Zaključak

Zadatak rada je funkcionalna e-commerce web aplikacija u ASP.NET Core MVC tehnologiji, sa katalogom proizvoda, korpom, sistemom korisničkih naloga i administracijom. Funkcionalnosti koje su implementirane su: pregled i pretraga proizvoda po kategorijama, korpa za kupovinu dostupna i gostima preko sesije, registracija/prijava i autorizacija po ulogama, checkout ograničen na prijavljene korisnike, istorija porudžbina, kao i kompletan administratorski panel (proizvodi, porudžbine, korisnici, statistika).

U celini, projekat nam je dao praktično iskustvo sa slojevitom MVC arhitekturom, radom sa sesijom kao privremenim skladištem stanja, autorizacijom po ulogama i integracijom modernog frontend alata (Tailwind CSS) u .NET build proces.

7. Literatura

1. Microsoft Learn - ASP.NET Core MVC dokumentacija: <https://learn.microsoft.com/aspnet/core/mvc/overview>
2. Hero Icons - ikonice korišćene za sajt: <https://heroicons.com/>
3. Stock Images - stock slike korišćene: <https://unsplash.com/>
4. Microsoft Learn - ASP.NET Core Identity dokumentacija: <https://learn.microsoft.com/aspnet/core/security/authentication/identity>
5. Tailwind CSS - zvanična dokumentacija (v4): <https://tailwindcss.com/docs>
6. Tailwind CSS Blog - najava verzije 4: <https://tailwindcss.com/blog/tailwindcss-v4>
7. Tailwind Plus Elements dokumentacija: <https://tailwindcss.com/plus/ui-elements/documentation>
8. Tailwind UI Blockovi: <https://tailwindcss.com/plus/ui-blocks/application-ui>